NPR
College of Engineering and Technology
(An Autonomous Institution)
NBA Accredited (B.E. - CSE, ECE, EEE & Mechanical Engg.) | Accredited by NAAC with 'A' Grade
Recognized by UGC under 2 (f) | Approved by All India Council for Technical Education, New Delhi | Affiliated to Anna University, Chennai
Natham, Dindigul (Dist.) - 624 401, Tamil Nadu. | Web: www.nprcet.edu.in

DATABASE TECHNOLOGIES

LAB WORK

Name: KARTHIKEYAN.J

PROGRAM: Master of Computer Application

SUBJECT CODE: 26MC103

PROGRAM 1: DDL AND DML OPERATIONS

Aim

To create a Product table and perform DDL and DML operations using SQL commands.

Procedure

1. Create a database named ShopDB.
2. Create a Product table with suitable fields.
3. Insert product records into the table.
4. Display the records using SELECT.
5. Update the price of a product using UPDATE.
6. Delete a product using DELETE.
7. Add a new column using ALTER TABLE.
8. Display the table structure using DESC.
9. Remove all records using TRUNCATE.
10. Stop the program.

Source Code

```
-- Create Database Table
-- Oracle uses the connected user schema.
-- Create Product Table

CREATE TABLE Product (
   Product_ID NUMBER(5) PRIMARY KEY,
   Product_Name VARCHAR2(30),
   Category VARCHAR2(20),
   Price NUMBER(10)
);

-- Insert Product Records
INSERT INTO Product VALUES (101, 'Keyboard', 'Electronics', 800);

INSERT INTO Product VALUES (102, 'Mouse', 'Electronics', 500);

INSERT INTO Product VALUES (103, 'Notebook', 'Stationery', 100);

-- Display Initial Records
SELECT * FROM Product;
```

```
-- Update Product Price
UPDATE Product
SET Price = 900
WHERE Product_ID = 101;

-- Display Records After UPDATE
SELECT * FROM Product;

-- Delete a Product
DELETE FROM Product
WHERE Product_ID = 103;

-- Display Records After DELETE
SELECT * FROM Product;

-- Add a New Column
ALTER TABLE Product
ADD Stock NUMBER(5);

-- Display Table Structure
DESC Product;

-- Update Stock Values
UPDATE Product
SET Stock = 20
WHERE Product_ID = 101;

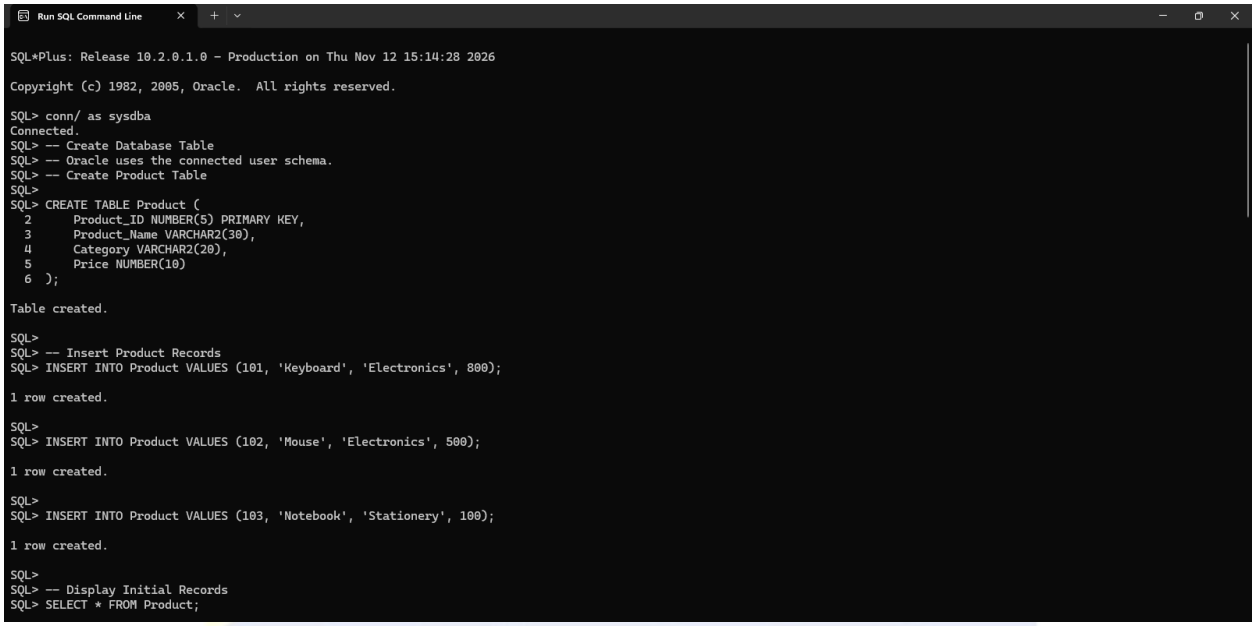
UPDATE Product
SET Stock = 35
WHERE Product_ID = 102;

-- Display Records After ALTER TABLE
SELECT * FROM Product;

-- Remove All Records
TRUNCATE TABLE Product;

-- Display Empty Table
SELECT * FROM Product;

-- Stop the Program
EXIT;
```



```
Run SQL Command Line

SQL*Plus: Release 10.2.0.1.0 - Production on Thu Nov 12 15:14:28 2026

Copyright (c) 1982, 2005, Oracle.  All rights reserved.

SQL> conn/ as sysdba
Connected.
SQL> -- Create Database Table
SQL> -- Oracle uses the connected user schema.
SQL> -- Create Product Table
SQL>
SQL> CREATE TABLE Product (
  2      Product_ID NUMBER(5) PRIMARY KEY,
  3      Product_Name VARCHAR2(30),
  4      Category VARCHAR2(20),
  5      Price NUMBER(10)
  6  );

Table created.

SQL>
SQL> -- Insert Product Records
SQL> INSERT INTO Product VALUES (101, 'Keyboard', 'Electronics', 800);

1 row created.

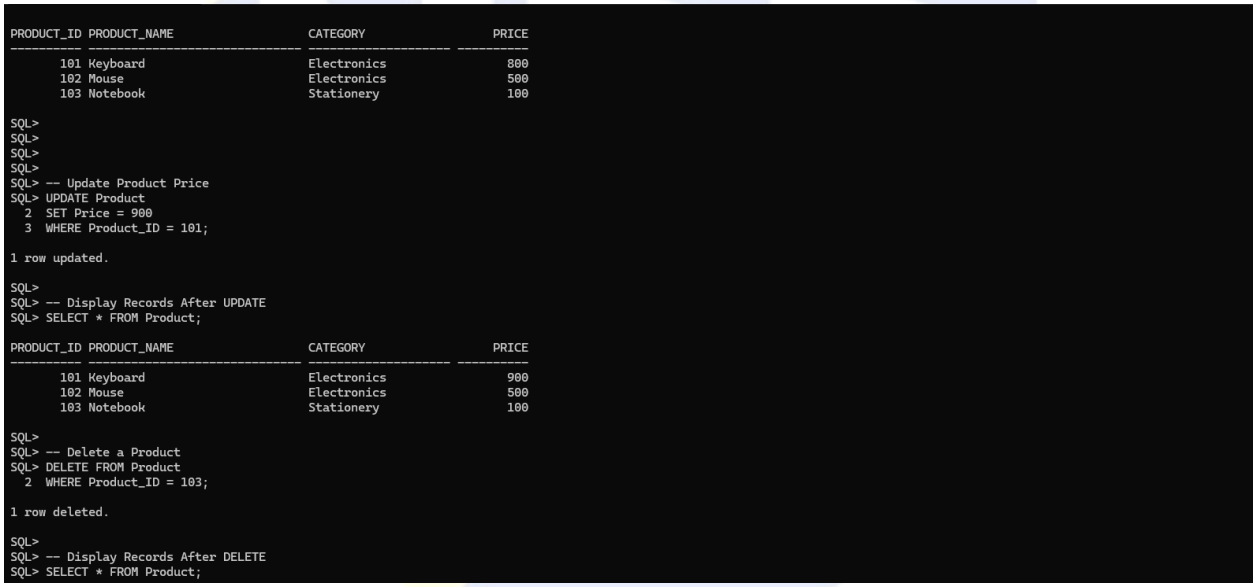
SQL>
SQL> INSERT INTO Product VALUES (102, 'Mouse', 'Electronics', 500);

1 row created.

SQL>
SQL> INSERT INTO Product VALUES (103, 'Notebook', 'Stationery', 100);

1 row created.

SQL>
SQL> -- Display Initial Records
SQL> SELECT * FROM Product;
```



```
PRODUCT_ID PRODUCT_NAME                   CATEGORY                 PRICE
---------- ------------------------------ -------------------- ----------
       101 Keyboard                       Electronics                 800
       102 Mouse                          Electronics                 500
       103 Notebook                       Stationery                  100

SQL>
SQL>
SQL>
SQL>
SQL> -- Update Product Price
SQL> UPDATE Product
  2  SET Price = 900
  3  WHERE Product_ID = 101;

1 row updated.

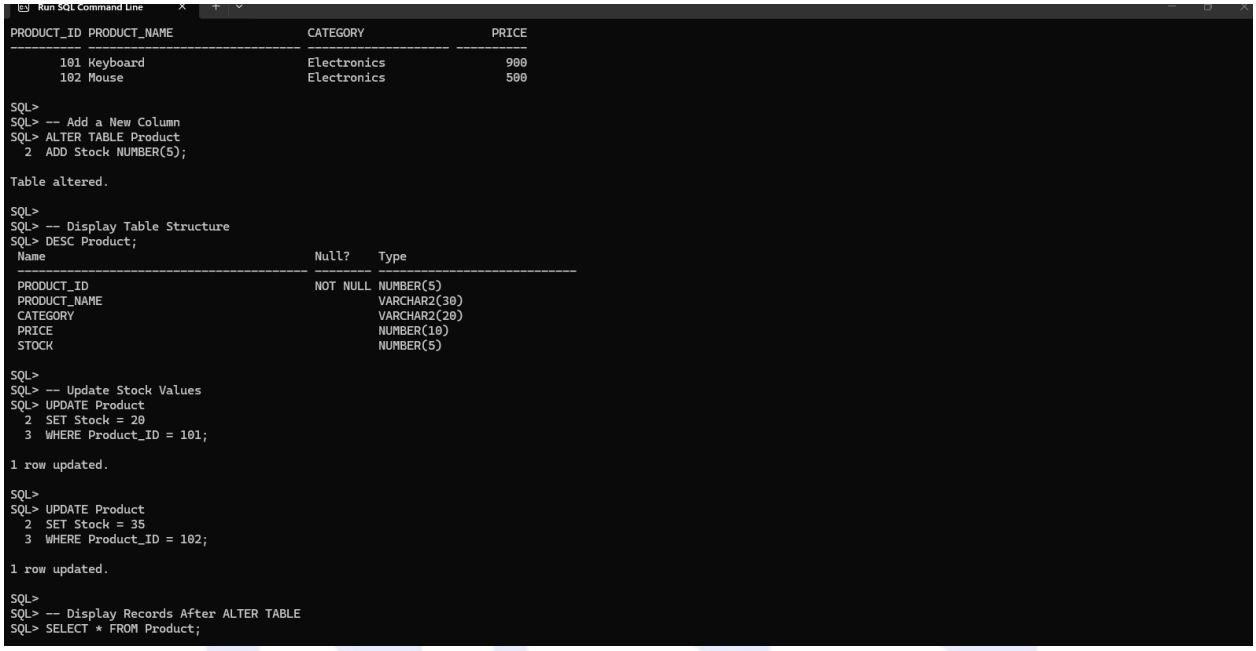
SQL>
SQL> -- Display Records After UPDATE
SQL> SELECT * FROM Product;

PRODUCT_ID PRODUCT_NAME                   CATEGORY                 PRICE
---------- ------------------------------ -------------------- ----------
       101 Keyboard                       Electronics                 900
       102 Mouse                          Electronics                 500
       103 Notebook                       Stationery                  100

SQL>
SQL> -- Delete a Product
SQL> DELETE FROM Product
  2  WHERE Product_ID = 103;

1 row deleted.

SQL>
SQL> -- Display Records After DELETE
SQL> SELECT * FROM Product;
```



```
Run SQL Command Line

PRODUCT_ID PRODUCT_NAME                    CATEGORY                 PRICE
---------- ------------------------------ -------------------- ----------
       101 Keyboard                       Electronics                 900
       102 Mouse                          Electronics                 500

SQL>
SQL> -- Add a New Column
SQL> ALTER TABLE Product
  2  ADD Stock NUMBER(5);

Table altered.

SQL>
SQL> -- Display Table Structure
SQL> DESC Product;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 PRODUCT_ID                                NOT NULL NUMBER(5)
 PRODUCT_NAME                                       VARCHAR2(30)
 CATEGORY                                           VARCHAR2(20)
 PRICE                                              NUMBER(10)
 STOCK                                              NUMBER(5)

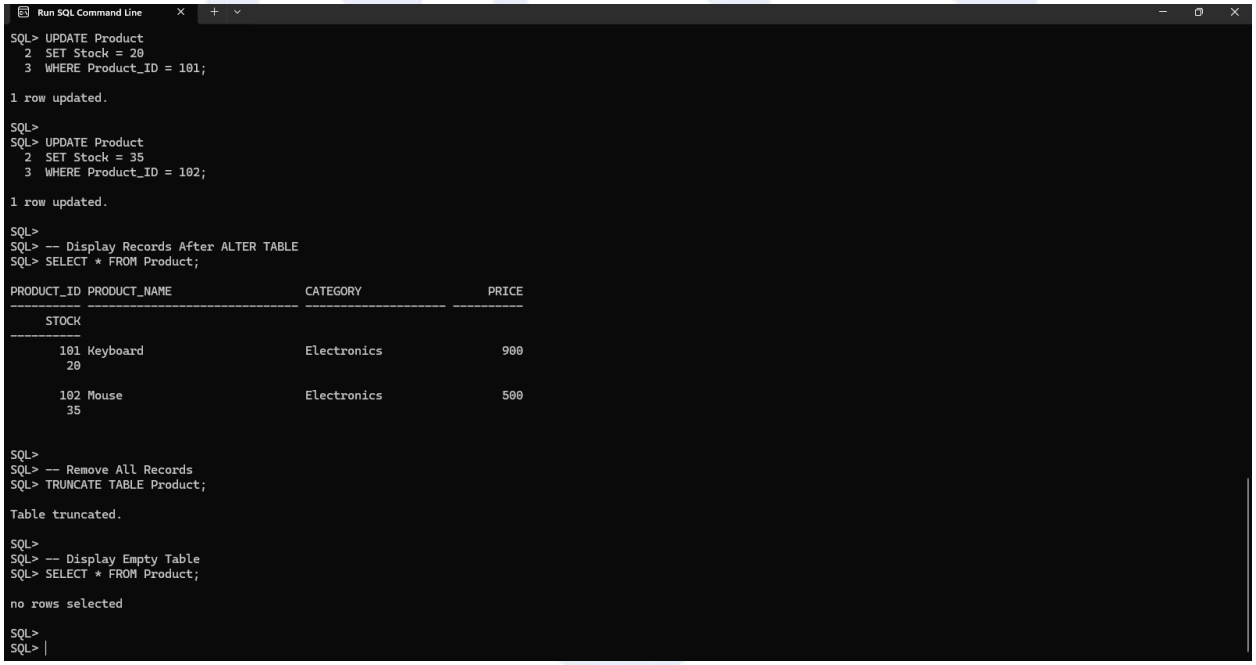
SQL>
SQL> -- Update Stock Values
SQL> UPDATE Product
  2  SET Stock = 20
  3  WHERE Product_ID = 101;

1 row updated.

SQL>
SQL> UPDATE Product
  2  SET Stock = 35
  3  WHERE Product_ID = 102;

1 row updated.

SQL>
SQL> -- Display Records After ALTER TABLE
SQL> SELECT * FROM Product;
```



```
Run SQL Command Line

SQL> UPDATE Product
  2  SET Stock = 20
  3  WHERE Product_ID = 101;

1 row updated.

SQL>
SQL> UPDATE Product
  2  SET Stock = 35
  3  WHERE Product_ID = 102;

1 row updated.

SQL>
SQL> -- Display Records After ALTER TABLE
SQL> SELECT * FROM Product;

PRODUCT_ID PRODUCT_NAME                    CATEGORY                 PRICE
---------- ------------------------------ -------------------- ----------
     STOCK
----------
       101 Keyboard                       Electronics                 900
        20

       102 Mouse                          Electronics                 500
        35


SQL>
SQL> -- Remove All Records
SQL> TRUNCATE TABLE Product;

Table truncated.

SQL>
SQL> -- Display Empty Table
SQL> SELECT * FROM Product;

no rows selected

SQL>
SQL>
```

Output

Initial records

Product_ID	Product_Name	Category	Price
101	Keyboard	Electronics	800
102	Mouse	Electronics	500
103	Notebook	Stationery	100

After UPDATE

Product_ID	Product_Name	Category	Price
101	Keyboard	Electronics	900
102	Mouse	Electronics	500
103	Notebook	Stationery	100

After DELETE

Product_ID	Product_Name	Category	Price
101	Keyboard	Electronics	900
102	Mouse	Electronics	500

After ALTER TABLE, the Stock column is added. After TRUNCATE, the table contains zero records.

Product_ID	Product_Name	Category	Price	Stock
101	Keyboard	Electronics	900	20
102	Mouse	Electronics	500	35

Result

Thus, DDL and DML operations were performed successfully on the Product table.

PROGRAM 2: FOREIGN KEY AND REFERENTIAL INTEGRITY

Aim

To create related Customer and Orders tables and implement foreign key constraints to maintain referential integrity.

Procedure

1. Create a database named SalesDB.
2. Create a Customer table with Customer_ID as the primary key.
3. Create an Orders table with Order_ID as the primary key and Customer_ID as a foreign key.
4. Insert customer records.
5. Insert orders for existing customers.
6. Display the records from both tables.
7. Attempt to insert an order for a customer who does not exist.
8. Verify the foreign key constraint.
9. Stop the program.

Source Code

```
-- Create Customer Table
CREATE TABLE Customer (
    Customer_ID NUMBER(5) PRIMARY KEY,
    Customer_Name VARCHAR2(30) NOT NULL,
    City VARCHAR2(30)
);

-- Create Orders Table
CREATE TABLE Orders (
    Order_ID NUMBER(5) PRIMARY KEY,
    Order_Date DATE,
    Amount NUMBER(10),
    Customer_ID NUMBER(5),
    CONSTRAINT fk_customer
    FOREIGN KEY (Customer_ID)
    REFERENCES Customer(Customer_ID)
);


-- Insert Customer Records
```

```
INSERT INTO Customer
VALUES (1, 'Arun', 'Chennai');

INSERT INTO Customer
VALUES (2, 'Priya', 'Madurai');

INSERT INTO Customer
VALUES (3, 'Kumar', 'Coimbatore');

-- Insert Orders Records
INSERT INTO Orders
VALUES (101, TO_DATE('10-01-2026','DD-MM-YYYY'), 2500, 1);

INSERT INTO Orders
VALUES (102, TO_DATE('12-01-2026','DD-MM-YYYY'), 1800, 2);

INSERT INTO Orders
VALUES (103, TO_DATE('15-01-2026','DD-MM-YYYY'), 3200, 1);

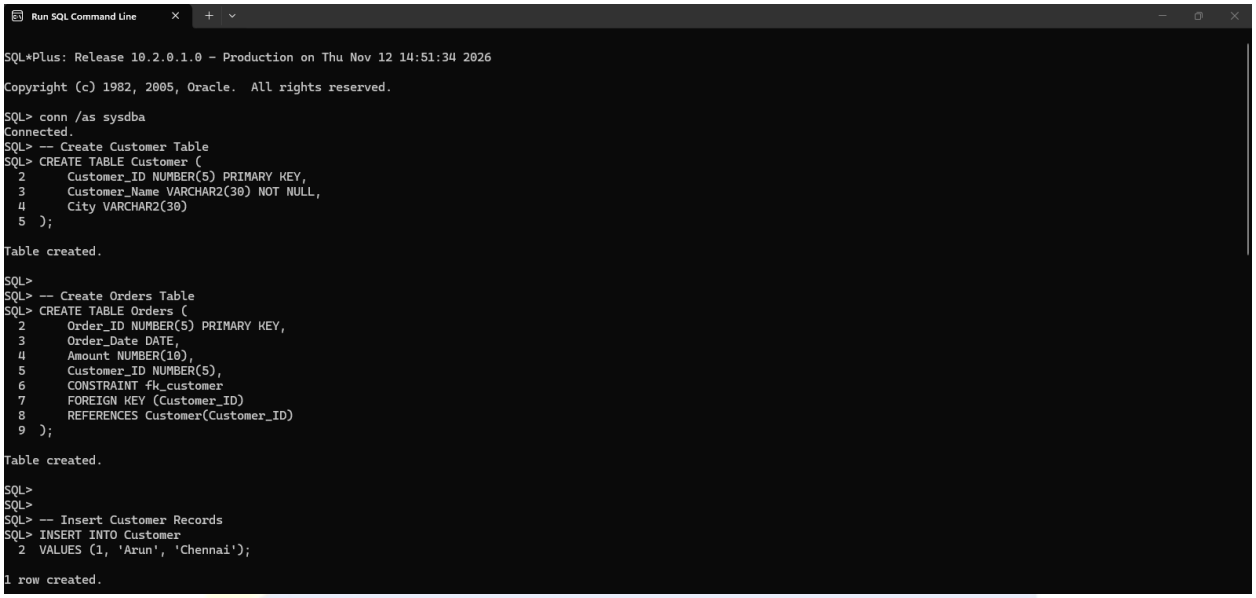
INSERT INTO Orders
VALUES (104, TO_DATE('20-01-2026','DD-MM-YYYY'), 1500, 3);

-- Display Customer Table
SELECT * FROM Customer;

-- Display Orders Table
SELECT * FROM Orders;

-- Test Referential Integrity
INSERT INTO Orders
VALUES (105, TO_DATE('25-01-2026','DD-MM-YYYY'), 2000, 5);

-- Exit
EXIT;
```



```
Run SQL Command Line

SQL*Plus: Release 10.2.0.1.0 - Production on Thu Nov 12 14:51:34 2026

Copyright (c) 1982, 2005, Oracle.  All rights reserved.

SQL> conn /as sysdba
Connected.
SQL> -- Create Customer Table
SQL> CREATE TABLE Customer (
  2      Customer_ID NUMBER(5) PRIMARY KEY,
  3      Customer_Name VARCHAR2(30) NOT NULL,
  4      City VARCHAR2(30)
  5  );

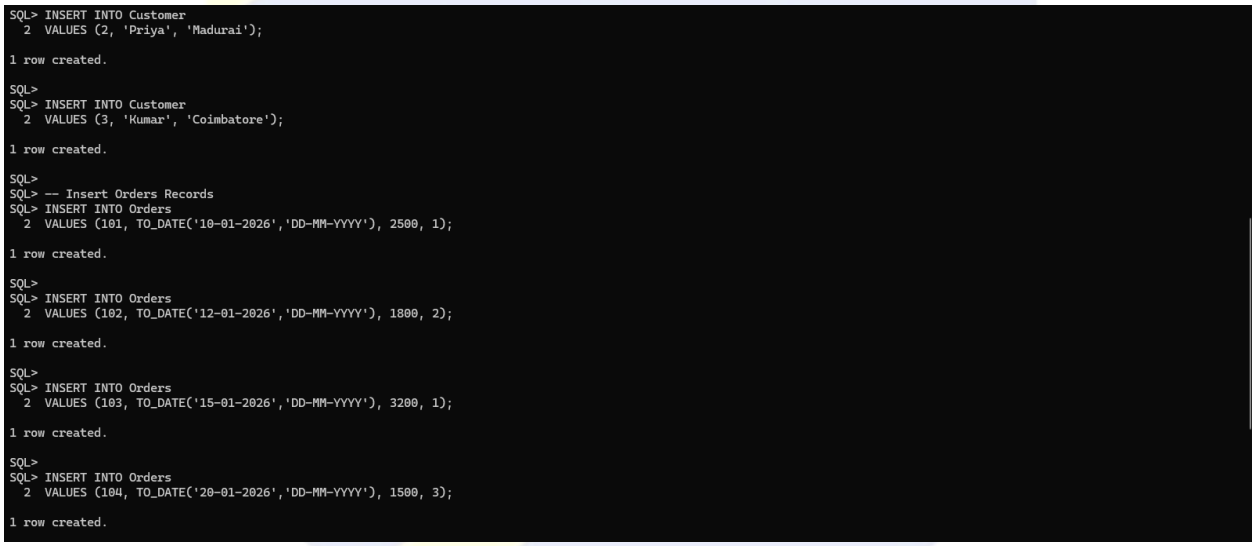
Table created.

SQL>
SQL> -- Create Orders Table
SQL> CREATE TABLE Orders (
  2      Order_ID NUMBER(5) PRIMARY KEY,
  3      Order_Date DATE,
  4      Amount NUMBER(10),
  5      Customer_ID NUMBER(5),
  6      CONSTRAINT fk_customer
  7      FOREIGN KEY (Customer_ID)
  8      REFERENCES Customer(Customer_ID)
  9  );

Table created.

SQL>
SQL>
SQL> -- Insert Customer Records
SQL> INSERT INTO Customer
  2  VALUES (1, 'Arun', 'Chennai');

1 row created.
```



```
SQL> INSERT INTO Customer
  2  VALUES (2, 'Priya', 'Madurai');

1 row created.

SQL>
SQL> INSERT INTO Customer
  2  VALUES (3, 'Kumar', 'Coimbatore');

1 row created.

SQL>
SQL> -- Insert Orders Records
SQL> INSERT INTO Orders
  2  VALUES (101, TO_DATE('10-01-2026','DD-MM-YYYY'), 2500, 1);

1 row created.

SQL>
SQL> INSERT INTO Orders
  2  VALUES (102, TO_DATE('12-01-2026','DD-MM-YYYY'), 1800, 2);

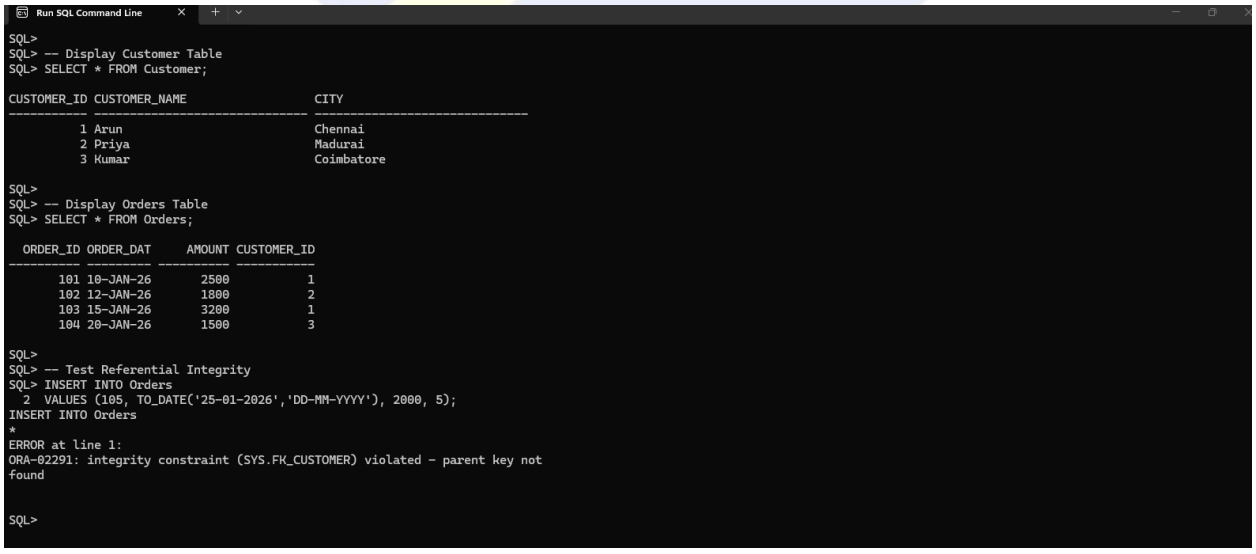
1 row created.

SQL>
SQL> INSERT INTO Orders
  2  VALUES (103, TO_DATE('15-01-2026','DD-MM-YYYY'), 3200, 1);

1 row created.

SQL>
SQL> INSERT INTO Orders
  2  VALUES (104, TO_DATE('20-01-2026','DD-MM-YYYY'), 1500, 3);

1 row created.
```



```
Run SQL Command Line

SQL>
SQL> -- Display Customer Table
SQL> SELECT * FROM Customer;

CUSTOMER_ID CUSTOMER_NAME                  CITY
----------- ------------------------------ ------------------------------
          1 Arun                           Chennai
          2 Priya                          Madurai
          3 Kumar                          Coimbatore

SQL>
SQL> -- Display Orders Table
SQL> SELECT * FROM Orders;

  ORDER_ID ORDER_DAT     AMOUNT CUSTOMER_ID
---------- --------- ---------- -----------
       101 10-JAN-26       2500           1
       102 12-JAN-26       1800           2
       103 15-JAN-26       3200           1
       104 20-JAN-26       1500           3

SQL>
SQL> -- Test Referential Integrity
SQL> INSERT INTO Orders
  2  VALUES (105, TO_DATE('25-01-2026','DD-MM-YYYY'), 2000, 5);
INSERT INTO Orders
*
ERROR at line 1:
ORA-02291: integrity constraint (SYS.FK_CUSTOMER) violated - parent key not
found


SQL>
```

Output

Customer Table

Customer_ID	Customer_Name	City
1	Arun	Chennai
2	Priya	Madurai
3	Kumar	Coimbatore

Orders Table

Order_ID	Order_Date	Amount	Customer_ID
101	2026-01-10	2500	1
102	2026-01-12	1800	2
103	2026-01-15	3200	1
104	2026-01-20	1500	3

Invalid insertion
The insertion of Order_ID 105 is rejected because Customer_ID 5 does not exist in the Customer table.

Result

Thus, foreign key constraints were implemented successfully, and referential integrity was maintained between the Customer and Orders tables.

PROGRAM 3: WHERE CLAUSE AND AGGREGATE FUNCTIONS

Aim

To retrieve book details using the WHERE clause and calculate aggregate values using COUNT(), SUM(), AVG(), MAX() and MIN().

Procedure

1. Create a Book table.
2. Insert sample book records.
3. Retrieve books with a price greater than 300 using WHERE.
4. Calculate the total number of books.
5. Calculate the total and average price.
6. Find the highest and lowest prices using aggregate functions.
7. Display the results.
8. Stop the program.

Source Code

```
CREATE TABLE Book (
Book_ID NUMBER(5) PRIMARY KEY,
Book_Name VARCHAR2(40),
Category VARCHAR2(20),
Price NUMBER(10)
);
-- Insert Book Records
INSERT INTO Book VALUES (1, 'SQL Basics', 'Database', 450);
INSERT INTO Book VALUES (2, 'Python Guide', 'Programming', 600);
INSERT INTO Book VALUES (3, 'Java Basics', 'Programming', 500);
INSERT INTO Book VALUES (4, 'Web Design', 'Web', 300);
INSERT INTO Book VALUES (5, 'DBMS Guide', 'Database', 400);
-- Display All Book Records
SELECT * FROM Book;
-- Display Books Priced Above 300
SELECT *
FROM Book
```

```
WHERE Price > 300;
-- Aggregate Functions for Database Category
SELECT
COUNT(*) AS Total_Books,
SUM(Price) AS Total_Price,
AVG(Price) AS Average_Price,
MAX(Price) AS Highest_Price,
MIN(Price) AS Lowest_Price
FROM Book
WHERE Category = 'Database';


-- Stop the Program
EXIT;
```

```
SQL> CREATE TABLE Book (
  2  Book_ID NUMBER(5) PRIMARY KEY,
  3  Book_Name VARCHAR2(40),
  4  Category VARCHAR2(20),
  5  Price NUMBER(10)
  6  );

Table created.

SQL> -- Insert Book Records
SQL> INSERT INTO Book VALUES (1, 'SQL Basics', 'Database', 450);

1 row created.

SQL> INSERT INTO Book VALUES (2, 'Python Guide', 'Programming', 600);

1 row created.

SQL> INSERT INTO Book VALUES (3, 'Java Basics', 'Programming', 500);

1 row created.

SQL> INSERT INTO Book VALUES (4, 'Web Design', 'Web', 300); INSERT INTO Book VALUES (5, 'DBMS Guide', 'Database', 400);
INSERT INTO Book VALUES (4, 'Web Design', 'Web', 300); INSERT INTO Book VALUES (5, 'DBMS Guide', 'Database', 400)
                                                       *
ERROR at line 1:
ORA-00911: invalid character


SQL> INSERT INTO Book VALUES (4, 'Web Design', 'Web', 300);

1 row created.

SQL> INSERT INTO Book VALUES (5, 'DBMS Guide', 'Database', 400);
```

```
SQL> INSERT INTO Book VALUES (5, 'DBMS Guide', 'Database', 400);

1 row created.

SQL> SELECT * FROM Book;

   BOOK_ID BOOK_NAME                                CATEGORY
---------- ---------------------------------------- --------------------
     PRICE
----------
         1 SQL Basics                               Database
       450

         2 Python Guide                             Programming
       600

         3 Java Basics                              Programming
       500


   BOOK_ID BOOK_NAME                                CATEGORY
---------- ---------------------------------------- --------------------
     PRICE
----------
         4 Web Design                               Web
       300

         5 DBMS Guide                               Database
       400


SQL> SELECT *
  2  FROM Book
  3  WHERE Price > 300;

   BOOK_ID BOOK_NAME                                CATEGORY
```

```
SQL> SELECT *
  2  FROM Book
  3  WHERE Price > 300;

   BOOK_ID BOOK_NAME                                CATEGORY
---------- ---------------------------------------- --------------------
     PRICE
----------
         1 SQL Basics                               Database
       450

         2 Python Guide                             Programming
       600

         3 Java Basics                              Programming
       500


   BOOK_ID BOOK_NAME                                CATEGORY
---------- ---------------------------------------- --------------------
     PRICE
----------
         5 DBMS Guide                               Database
       400


SQL> SELECT
  2  COUNT(*) AS Total_Books,
  3  SUM(Price) AS Total_Price,
  4  AVG(Price) AS Average_Price,
  5  MAX(Price) AS Highest_Price,
  6  MIN(Price) AS Lowest_Price
  7  FROM Book
  8  WHERE Category = 'Database';
```

```
SQL> SELECT
  2  COUNT(*) AS Total_Books,
  3  SUM(Price) AS Total_Price,
  4  AVG(Price) AS Average_Price,
  5  MAX(Price) AS Highest_Price,
  6  MIN(Price) AS Lowest_Price
  7  FROM Book
  8  WHERE Category = 'Database';

TOTAL_BOOKS TOTAL_PRICE AVERAGE_PRICE HIGHEST_PRICE LOWEST_PRICE
----------- ----------- ------------- ------------- ------------
          2         850           425           450          400

SQL>
```

Output

Book_ID	Book_Name	Category	Price
1	SQL Basics	Database	450
2	Python Guide	Programming	600
3	Java Basics	Programming	500
4	WebDesign	Web	300
5	DBMS Guide	Database	400

Books priced above 300

Book_ID	Book_Name	Category	Price
1	SQL Basics	Database	450
2	Python Guide	Programming	600
3	Java Basics	Programming	500
5	DBMS Guide	Database	400

Aggregate Function Output

Total_Books	Total_Price	Average_Price	Highest_Price	Lowest_Price
2	850	425	450	400

Result

Thus, the WHERE clause and aggregate functions were executed successfully to retrieve and summarize book details.

PROGRAM 4: SUBQUERIES AND SIMPLE JOINS

Aim

To execute SQL queries using the WHERE clause, subquery with IN, INNER JOIN and LEFT JOIN using Library and Publisher details.

Procedure

1. Create a Publisher table and a LibraryBook table.
2. Insert records into both tables.
3. Retrieve books with a price greater than 400.
4. Use a subquery with IN to retrieve books from selected publishers.
5. Perform an INNER JOIN to display books with matching publisher details.
6. Perform a LEFT JOIN to display all books, including books without a matching publisher.
7. Display the results.
8. Stop the program.

Source Code

```
-- Create Publisher Table
CREATE TABLE Publisher (
   Publisher_ID NUMBER(5) PRIMARY KEY,
   Publisher_Name VARCHAR2(30)
);

-- Create LibraryBook Table
CREATE TABLE LibraryBook (
   Book_ID NUMBER(5) PRIMARY KEY,
   Book_Name VARCHAR2(30),
   Price NUMBER(10),
   Publisher_ID NUMBER(5)
);

-- Insert Publisher Records
INSERT INTO Publisher VALUES (10, 'Sun Publications');

INSERT INTO Publisher VALUES (20, 'Tech Publications');

INSERT INTO Publisher VALUES (30, 'Global Books');
```

```
-- Insert LibraryBook Records
INSERT INTO LibraryBook VALUES (1, 'SQL Guide', 450, 10);

INSERT INTO LibraryBook VALUES (2, 'Python Guide', 600, 20);

INSERT INTO LibraryBook VALUES (3, 'Java Guide', 500, 10);

INSERT INTO LibraryBook VALUES (4, 'Web Design', 350, 30);

INSERT INTO LibraryBook VALUES (5, 'C Programming', 400, NULL);

-- Display Publisher Table
SELECT * FROM Publisher;

-- Display LibraryBook Table
SELECT * FROM LibraryBook;

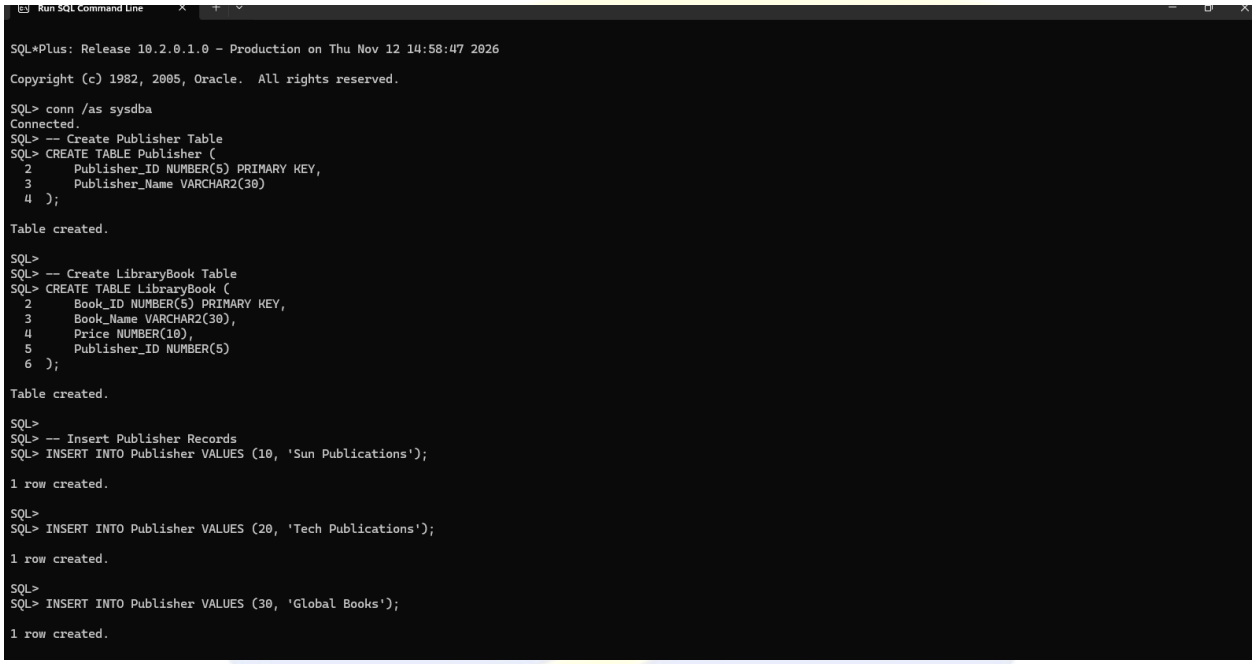
-- 1. WHERE Clause
SELECT *
FROM LibraryBook
WHERE Price > 400;

-- 2. Subquery Using IN
SELECT Book_Name, Price
FROM LibraryBook
WHERE Publisher_ID IN (
   SELECT Publisher_ID
   FROM Publisher
   WHERE Publisher_Name IN
   ('Sun Publications', 'Tech Publications')
);

-- 3. INNER JOIN
SELECT LibraryBook.Book_Name,
      Publisher.Publisher_Name
FROM LibraryBook
INNER JOIN Publisher
ON LibraryBook.Publisher_ID = Publisher.Publisher_ID;
```

```
-- 4. LEFT JOIN
SELECT LibraryBook.Book_Name,
       Publisher.Publisher_Name
FROM LibraryBook
LEFT JOIN Publisher
ON LibraryBook.Publisher_ID = Publisher.Publisher_ID;

-- Stop the Program
EXIT;
```



```
Run SQL Command Line

SQL>
SQL> INSERT INTO LibraryBook VALUES (2, 'Python Guide', 600, 20);

1 row created.

SQL>
SQL> INSERT INTO LibraryBook VALUES (3, 'Java Guide', 500, 10);

1 row created.

SQL>
SQL> INSERT INTO LibraryBook VALUES (4, 'Web Design', 350, 30);

1 row created.

SQL>
SQL> INSERT INTO LibraryBook VALUES (5, 'C Programming', 400, NULL);

1 row created.

SQL>
SQL> -- Display Publisher Table
SQL> SELECT * FROM Publisher;

PUBLISHER_ID PUBLISHER_NAME
------------ ------------------------------
          10 Sun Publications
          20 Tech Publications
          30 Global Books

SQL>
SQL> -- Display LibraryBook Table
SQL> SELECT * FROM LibraryBook;

   BOOK_ID BOOK_NAME                       PRICE PUBLISHER_ID
---------- ------------------------------ ---------- ------------
         1 SQL Guide                         450           10
         2 Python Guide                      600           20
         3 Java Guide                        500           10
         4 Web Design                        350           30
         5 C Programming                     400
```

```
Run SQL Command Line

SQL>
SQL> -- 1. WHERE Clause
SQL> SELECT *
  2  FROM LibraryBook
  3  WHERE Price > 400;

   BOOK_ID BOOK_NAME                       PRICE PUBLISHER_ID
---------- ------------------------------ ---------- ------------
         1 SQL Guide                         450           10
         2 Python Guide                      600           20
         3 Java Guide                        500           10

SQL>
SQL> -- 2. Subquery Using IN
SQL> SELECT Book_Name, Price
  2  FROM LibraryBook
  3  WHERE Publisher_ID IN (
  4      SELECT Publisher_ID
  5      FROM Publisher
  6      WHERE Publisher_Name IN
  7      ('Sun Publications', 'Tech Publications')
  8  );

BOOK_NAME                       PRICE
------------------------------ ----------
SQL Guide                         450
Python Guide                      600
Java Guide                        500

SQL>
SQL> -- 3. INNER JOIN
SQL> SELECT LibraryBook.Book_Name,
  2         Publisher.Publisher_Name
  3  FROM LibraryBook
  4  INNER JOIN Publisher
  5  ON LibraryBook.Publisher_ID = Publisher.Publisher_ID;

BOOK_NAME                      PUBLISHER_NAME
------------------------------ ------------------------------
SQL Guide                      Sun Publications
Python Guide                   Tech Publications
Java Guide                     Sun Publications
```

```
SQL>
SQL> -- 3. INNER JOIN
SQL> SELECT LibraryBook.Book_Name,
  2         Publisher.Publisher_Name
  3  FROM LibraryBook
  4  INNER JOIN Publisher
  5  ON LibraryBook.Publisher_ID = Publisher.Publisher_ID;

BOOK_NAME                      PUBLISHER_NAME
------------------------------ ------------------------------
SQL Guide                      Sun Publications
Python Guide                   Tech Publications
Java Guide                     Sun Publications
Web Design                     Global Books

SQL>
SQL> -- 4. LEFT JOIN
SQL> SELECT LibraryBook.Book_Name,
  2         Publisher.Publisher_Name
  3  FROM LibraryBook
  4  LEFT JOIN Publisher
  5  ON LibraryBook.Publisher_ID = Publisher.Publisher_ID;

BOOK_NAME                      PUBLISHER_NAME
------------------------------ ------------------------------
Java Guide                     Sun Publications
SQL Guide                      Sun Publications
Python Guide                   Tech Publications
Web Design                     Global Books
C Programming

SQL>
SQL>
```

Output

1. WHERE Clause

Book_ID	Book_Name	Price	Publisher_ID
1	SQL Guide	450	10
2	Python Guide	600	20
3	Java Guide	500	10

2. Subquery using IN

Book_Name	Price
SQL Guide	450
Python Guide	600
Java Guide	500

3. INNER JOIN

Book_Name	Publisher_Name
SQL Guide	Sun Publications
Python Guide	Tech Publications
Java Guide	Sun Publications
Web Design	Global Books

4. LEFT JOIN

Book_Name	Publisher_Name
SQL Guide	Sun Publications
Python Guide	Tech Publications
Java Guide	Sun Publications
Web Design	Global Books
C Programming	NULL

Result

Thus, the WHERE clause, subquery with IN, INNER JOIN and LEFT JOIN were executed successfully to retrieve library book and publisher details.

PROGRAM 5: NATURAL JOIN, EQUI JOIN AND OUTER JOINS

Aim

To perform Natural Join, Equi Join, Left Outer Join and Right Outer Join using Vehicle and VehicleType tables.

Procedure

1. Create Vehicle and VehicleType tables.
2. Insert sample vehicle and vehicle type records.
3. Perform a NATURAL JOIN using the common Type_ID column.
4. Perform an EQUI JOIN using the equality operator.
5. Perform a LEFT OUTER JOIN to display all vehicles and their types.
6. Perform a RIGHT OUTER JOIN to display all vehicle types and matching vehicles.
7. Display the results.
8. Stop the program.

Source Code

```
-- Create Vehicle Table
CREATE TABLE Vehicle (
   Vehicle_ID NUMBER(5) PRIMARY KEY,
   Vehicle_Name VARCHAR2(30),
   Type_ID NUMBER(5)
);

-- Create VehicleType Table
CREATE TABLE VehicleType (
   Type_ID NUMBER(5),
   Type_Name VARCHAR2(30)
);

-- Insert Vehicle Records
INSERT INTO Vehicle VALUES (1, 'Honda City', 10);

INSERT INTO Vehicle VALUES (2, 'Royal Enfield', 20);

INSERT INTO Vehicle VALUES (3, 'Hyundai i20', 10);

INSERT INTO Vehicle VALUES (4, 'Hero Cycle', 30);
```

```
-- Insert VehicleType Records
INSERT INTO VehicleType VALUES (10, 'Car');

INSERT INTO VehicleType VALUES (20, 'Bike');

INSERT INTO VehicleType VALUES (40, 'Bus');

-- Display Vehicle Table
SELECT * FROM Vehicle;

-- Display VehicleType Table
SELECT * FROM VehicleType;

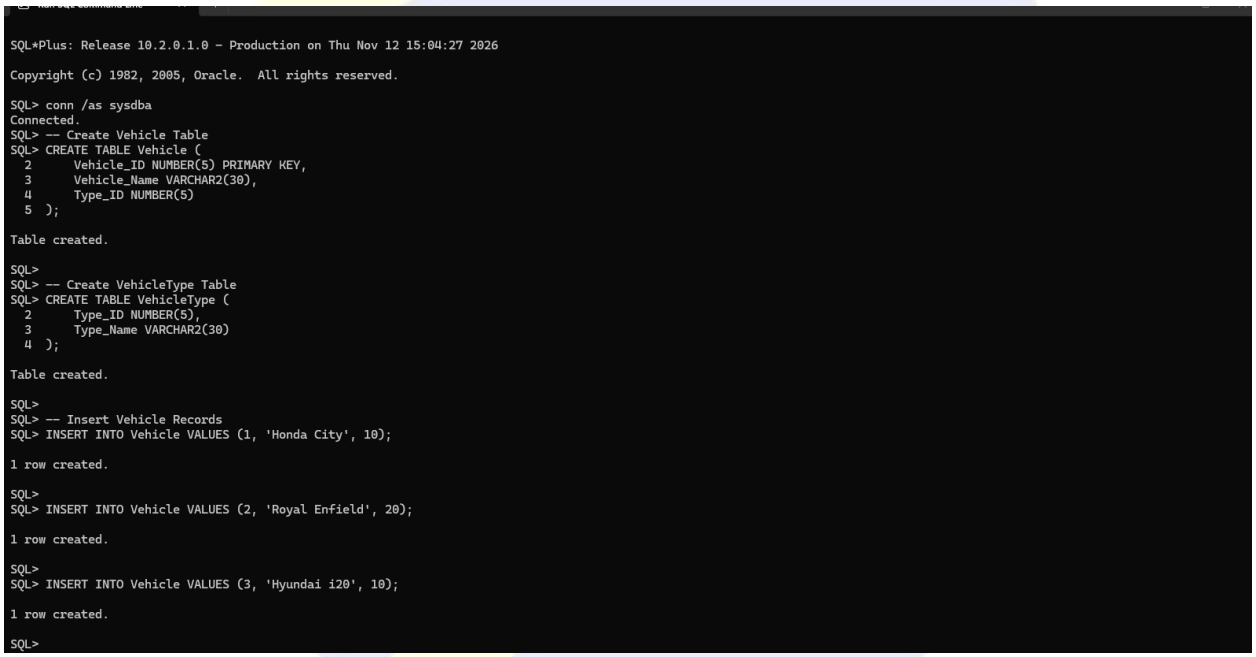
-- 1. NATURAL JOIN
SELECT *
FROM Vehicle
NATURAL JOIN VehicleType;

-- 2. EQUI JOIN
SELECT Vehicle.Vehicle_ID,
       Vehicle.Vehicle_Name,
       Vehicle.Type_ID,
       VehicleType.Type_Name
FROM Vehicle
JOIN VehicleType
ON Vehicle.Type_ID = VehicleType.Type_ID;

-- 3. LEFT OUTER JOIN
SELECT Vehicle.Vehicle_ID,
       Vehicle.Vehicle_Name,
       Vehicle.Type_ID,
       VehicleType.Type_Name
FROM Vehicle
LEFT OUTER JOIN VehicleType
ON Vehicle.Type_ID = VehicleType.Type_ID;
```

```
-- 4. RIGHT OUTER JOIN
SELECT Vehicle.Vehicle_ID,
       Vehicle.Vehicle_Name,
       VehicleType.Type_ID,
       VehicleType.Type_Name
FROM Vehicle
RIGHT OUTER JOIN VehicleType
ON Vehicle.Type_ID = VehicleType.Type_ID;

-- Stop the Program
EXIT;
```



Run SQL Command Line

```
SQL>
SQL> INSERT INTO Vehicle VALUES (3, 'Hyundai i20', 10);

1 row created.

SQL>
SQL> INSERT INTO Vehicle VALUES (4, 'Hero Cycle', 30);

1 row created.

SQL>
SQL> -- Insert VehicleType Records
SQL> INSERT INTO VehicleType VALUES (10, 'Car');

1 row created.

SQL>
SQL> INSERT INTO VehicleType VALUES (20, 'Bike');

1 row created.

SQL>
SQL> INSERT INTO VehicleType VALUES (40, 'Bus');

1 row created.

SQL>
SQL> -- Display Vehicle Table
SQL> SELECT * FROM Vehicle;

VEHICLE_ID VEHICLE_NAME                    TYPE_ID
---------- ------------------------------ ----------
         1 Honda City                             10
         2 Royal Enfield                          20
         3 Hyundai i20                            10
         4 Hero Cycle                             30
```

```
SQL>
SQL> -- Display VehicleType Table
SQL> SELECT * FROM VehicleType;

   TYPE_ID TYPE_NAME
---------- ------------------------------
        10 Car
        20 Bike
        40 Bus

SQL>
SQL> -- 1. NATURAL JOIN
SQL> SELECT *
  2  FROM Vehicle
  3  NATURAL JOIN VehicleType;

   TYPE_ID VEHICLE_ID VEHICLE_NAME
---------- ---------- ------------------------------
TYPE_NAME
------------------------------
        10          1 Honda City
Car

        20          2 Royal Enfield
Bike

        10          3 Hyundai i20
Car


SQL>
SQL> -- 2. EQUI JOIN
SQL> SELECT Vehicle.Vehicle_ID,
  2         Vehicle.Vehicle_Name,
  3         Vehicle.Type_ID,
  4         VehicleType.Type_Name
  5  FROM Vehicle
  6  JOIN VehicleType
  7  ON Vehicle.Type_ID = VehicleType.Type_ID;
```

```
VEHICLE_ID VEHICLE_NAME                    TYPE_ID
---------- ------------------------------ ----------
TYPE_NAME
------------------------------
         1 Honda City                             10
Car

         2 Royal Enfield                          20
Bike

         3 Hyundai i20                            10
Car


SQL>
SQL> -- 3. LEFT OUTER JOIN
SQL> SELECT Vehicle.Vehicle_ID,
  2         Vehicle.Vehicle_Name,
  3         Vehicle.Type_ID,
  4         VehicleType.Type_Name
  5  FROM Vehicle
  6  LEFT OUTER JOIN VehicleType
  7  ON Vehicle.Type_ID = VehicleType.Type_ID;

VEHICLE_ID VEHICLE_NAME                    TYPE_ID
---------- ------------------------------ ----------
TYPE_NAME
------------------------------
         3 Hyundai i20                            10
Car

         1 Honda City                             10
Car

         2 Royal Enfield                          20
Bike
```

```
VEHICLE_ID VEHICLE_NAME                    TYPE_ID
---------- ------------------------------ ----------
TYPE_NAME
------------------------------
         4 Hero Cycle                             30



SQL>
SQL> -- 4. RIGHT OUTER JOIN
SQL> SELECT Vehicle.Vehicle_ID,
  2         Vehicle.Vehicle_Name,
  3         VehicleType.Type_ID,
  4         VehicleType.Type_Name
  5  FROM Vehicle
  6  RIGHT OUTER JOIN VehicleType
  7  ON Vehicle.Type_ID = VehicleType.Type_ID;

VEHICLE_ID VEHICLE_NAME                    TYPE_ID
---------- ------------------------------ ----------
TYPE_NAME
------------------------------
         1 Honda City                             10
Car

         2 Royal Enfield                          20
Bike

         3 Hyundai i20                            10
Car


VEHICLE_ID VEHICLE_NAME                    TYPE_ID
---------- ------------------------------ ----------
TYPE_NAME
------------------------------
                                                  40
Bus
```

Output

1. Natural Join

Type_ID	Vehicle_ID	Vehicle_Name	Type_Name
10	1	Honda City	Car
20	2	Royal Enfield	Bike
10	3	Hyundai i20	Car

2. Equi Join

Vehicle_ID	Vehicle_Name	Type_ID	Type_Name
1	Honda City	10	Car
2	Royal Enfield	20	Bike
3	Hyundai i20	10	Car

3. Left Outer Join

Vehicle_ID	Vehicle_Name	Type_ID	Type_Name
1	Honda City	10	car
2	Royal Enfield	20	Bike
3	Hyundai i20	10	Car
4	Hero Cycle	30	NULL

4. Right Outer Join

Vehicle_ID	Vehicle_Name	Type_ID	Type_Name
1	Honda City	10	Car
2	Royal Enfield	20	Bike
3	Hyundai i20	10	Car
NULL	NULL	40	Bus

Result

Thus, Natural Join, Equi Join, Left Outer Join and Right Outer Join were executed successfully to retrieve vehicle and vehicle type details.